Sprint 8 · Reto «Duelo de modelos»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. TurisData ya sabe que se pueden predecir las cancelaciones de reservas (lo visteis en el Sprint 7). Ahora el gerente pregunta: **¿qué modelo ponemos en producción?** Duda entre tres cosas: *«¿el más preciso? ¿el que yo pueda entender? ¿el más barato de mantener?»*. Vuestro equipo tiene que **enfrentar varios modelos** (árboles, bosques, *boosting* y SVM), medirlos todos con las mismas reglas en una tabla de resultados (el *leaderboard*) y **elegir un campeón con argumentos**. La idea clave: en la vida real **el modelo más preciso no siempre es el mejor**; uno un poco peor, pero que se entiende y se mantiene fácilmente, puede ser la mejor decisión. Por eso se decide con **tres criterios**: rendimiento, explicabilidad y coste.

Cómo se trabaja. No hace falta experiencia previa: el cuaderno `S08_06` ya trae las herramientas para medir todos los modelos igual, y vosotros rellenáis los huecos paso a paso. Hay una regla de oro: **el test se usa una sola vez, solo con el campeón**; probar varios modelos en el test y quedarse con el mejor es hacer trampa. El trabajo completo son unos **150 minutos de equipo** (cinco pasos): en clase tenéis 10 minutos de planning (día 1), **75 minutos de reto + 35 de entrega, review y retro** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (el navegador, sin instalar nada) con las librerías `scikit-learn`, `xgboost` y `lightgbm`	Ejecutar el cuaderno del reto	Ya instaladas en Colab
El cuaderno del reto `S08_06_leaderboard_plantilla`	Medir todos los modelos igual, guardar `leaderboard_S08.csv`, calcular la puntuación global y evaluar el campeón en el test	Colab. Su código explicado
Cuadernos de modelos: `S08_01` árboles, `S08_02` bosques, `S08_03` *boosting* y `S08_04` SVM	Las pistas para cada familia de modelos	Colab; su código explicado
Cuaderno de búsqueda de hiperparámetros `S08_05`	La guía para el paso 3 (`GridSearchCV` y `RandomizedSearchCV`). Es el cuaderno **del reto** que se hace **en casa la tarde del 11/11** (≈ 40 min; para el paso 3 os bastan las secciones 2 y 3): venid con él hecho al paso 3	Colab; su código explicado
Presentaciones y apuntes	Repasar cómo funciona cada modelo: árboles, ensembles, SVM y apuntes	Carpeta `1_Teoria`

Recurso	Para qué lo usaréis	Dónde está
Plantilla de la *model card* `model_card_plantilla.md`	La «ficha técnica» del campeón, con sus apartados	En GitHub: *Raw* y copiar el texto
Plantilla del *leaderboard* `leaderboard_plantilla.csv`	Ver qué columnas tiene la tabla (el cuaderno ya crea la vuestra)	Solo para consultar
Los datos `reservas_turisdata.csv`	El problema: predecir la columna `cancelada`. Son reservas **sintéticas**	El cuaderno los descarga solo si no los encuentra
Cómo entregar en GitHub	Subir los ficheros a la carpeta `sprint-08/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender modelos y errores, y preparar la defensa	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son cinco pasos, de **150 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. El calendario da menos tiempo de reto del que suman los pasos: el día 1 solo hay 10 minutos (el planning, donde se acuerdan los pesos; los otros 10 son para escribirlos justificados al empezar el día 2) y el día 2 hay 75 minutos de reto, más la entrega, la review y la retro. Los 65 minutos que faltan se hacen al final del día 2, en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30); la subida puede hacerse hasta esa hora. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Paso 1 · Planning y pesos de los criterios (día 1, 20 minutos)

1. Repartir roles y tareas, y apuntarlas en el tablero.
2. Leer esta ficha y decidir hasta qué nivel queréis llegar (, o).
3. Acordar los **pesos** de los tres criterios con la «voz del cliente» (el docente) y justificarlos.
4. Escribir los pesos y su justificación en un sitio común: los usaréis en los pasos 4 y 5.

Cómo se hace

- **Los tres criterios.** Cada modelo se puntúa en tres cosas, y vosotros decidís cuánto vale cada una (los pesos suman 100 %):

Criterio	Qué mide	Peso (ejemplo)
Rendimiento	El AUC en validación cruzada, con su desviación (media ± desviación)	50 %
Explicabilidad	¿Puede el gerente entender **por qué** el modelo avisa? Se puntúa de 1 (caja negra) a 5 (se lee directamente)	30 %
Coste	Tiempo de entrenamiento y de predicción, tamaño del modelo y esfuerzo de mantenimiento	20 %

- **Justificarlos.** Los pesos son vuestros; lo importante es que tengan una razón. Preguntad al docente (que hace de cliente): «¿Necesita el gerente explicar a un cliente por qué se le avisa?», «¿hay poco presupuesto para mantener el modelo?». Ejemplo de frase: «Damos 30 % a la explicabilidad porque el gerente necesita entender por qué se avisa».
- **Repartir el trabajo.** Un reparto posible (con 3 personas, la persona D no existe y su tarea se reparte):

Persona	Tarea	Paso
A	Modelos de referencia, lineal, árbol y bosque en el cuaderno	2

Persona	Tarea	Paso
B	*Boosting*, SVM y primera búsqueda de hiperparámetros	3
C	Segunda búsqueda de hiperparámetros y análisis de sensibilidad	3 y 4
D (si hay 4)	*Model card* y README	5

Todos revisan el trabajo de otra persona. Para el cuaderno, usad **una sola copia compartida** (Compartir → editor) o una pantalla compartida, y no editéis la misma celda a la vez.

Palabras que vais a usar.

Palabra	Qué significa, en sencillo
Leaderboard	La tabla de clasificación de los modelos, con sus resultados
Familia de modelos	Un tipo de modelo: árbol, bosque, *boosting*, SVM, lineal…
Hiperparámetro	Un ajuste que hacéis vosotros **antes** de entrenar (la profundidad del árbol, el número de árboles…)
Validación cruzada	Evaluar varias veces con particiones distintas del *train* para tener una medida fiable
AUC	Nota de 0,5 (como el azar) a 1 (perfecto): cómo de bien separa las reservas que se cancelan de las que no
Explicabilidad	Poder decir **por qué** el modelo toma una decisión
Model card	La «ficha técnica» del modelo: usos, datos, métricas, límites y riesgos
Deriva	Cuando los clientes cambian su comportamiento con el tiempo y el modelo deja de funcionar bien

Qué deberíais ver. Los pesos escritos (por ejemplo, 50 / 30 / 20) con una frase de por qué, las tareas en el tablero y el nivel objetivo acordado.

Hecho cuando: los pesos suman 100 %, están justificados y cada persona sabe qué le toca.

Paso 2 · Primeros modelos en el leaderboard (día 2, 30 minutos)

1. Abrir `S08_06_leaderboard_plantilla` en Colab, guardar una copia y llamarla `reto_S08.ipynb`.
2. Ejecutar las celdas de preparación y de herramientas **sin cambiarlas**.
3. Registrar el modelo de referencia (*baseline*), la regresión logística, un árbol podado y un bosque.
4. Comprobar que la tabla tiene una fila por modelo.

Cómo se hace

- **Abrir y guardar una copia.** Abrid el cuaderno, pulsad *Archivo* → *Guardar una copia en Drive* y luego *Archivo* → *Cambiar nombre* para llamarla `reto_S08.ipynb`. Trabajad siempre en **vuestra copia**.
- **Ejecutar en orden.** Ejecutad las celdas de arriba abajo; la primera (*Preparación*) siempre antes que las demás. La celda **«Herramientas del leaderboard»** no se toca: es la que garantiza **mismos datos, mismos folds y misma métrica** para todos los modelos.
- **Rellenar los huecos.** Donde hay `# TODO` y `...`, escribid vosotros el código. Cada modelo se registra con el patrón `registrar(evaluar_modelo(nombre, familia, hacer_pipe(...), explicabilidad, notas))`. Las pistas de los cuatro de este paso:

Modelo (nombre)	Familia	Qué le pasáis a `hacer_pipe`	Explicabilidad
`baseline`	`referencia`	`DummyClassifier(strategy="prior")` (siempre predice lo más habitual)	5
`logistica`	`lineal`	`LogisticRegression(max_iter=1000)` con `escalar=True`	4

Modelo (nombre)	Familia	Qué le pasáis a `hacer_pipe`	Explicabilidad
`arbol_podado`	`arbol`	`DecisionTreeClassifier(min_samples_leaf=80, random_state=SEMILLA)`	5
`random_forest`	`ensemble_bagging`	`RandomForestClassifier(n_estimators=200, min_samples_leaf=5, random_state=SEMILLA, n_jobs=2)`	2

- **Dos tandas.** Los ejercicios 1 y 2 del cuaderno traen una comprobación (`assert`). La del ejercicio 2 pide 6 modelos, así que dará error hasta que estén también *boosting* y SVM (paso 3): es normal. Pistas de cuaderno: `S08_01` (árbol) y `S08_02` (bosque).

Qué deberíais ver. Una tabla ordenada por AUC con cuatro filas. El *baseline* sale con AUC cerca de 0,5 (el cuaderno lo comprueba) y el resto, por encima. Cada fila trae AUC medio, desviación, tiempos, tamaño y explicabilidad. En el panel de archivos de Colab (icono de la carpeta, a la izquierda) aparece `leaderboard_S08.csv`.

Hecho cuando: el leaderboard tiene `baseline`, `logistica`, `arbol_podado` y `random_forest`, y cada persona sabe explicar qué mide cada columna.

Paso 3 · *Boosting*, SVM y búsqueda de hiperparámetros (día 2, 25 minutos)

1. Registrar un modelo de *boosting* y una SVM.
2. Buscar hiperparámetros en **dos familias** (rejilla o aleatoria), solo con train y validación cruzada, sin pasar de 5 minutos cada búsqueda.
3. Registrar los mejores modelos y actualizar el `catalogo` del cuaderno.
4. Comprobar que hay entre 6 y 8 modelos, de al menos 4 familias.

Cómo se hace

- **Los dos modelos que faltan.** Mismo patrón que en el paso 2 (pistas en `S08_03` y `S08_04`):

Modelo (nombre)	Familia	Qué le pasáis a `hacer_pipe`	Explicabilidad
`lightgbm`	`ensemble_boosting`	`lgb.LGBMClassifier(n_estimators=100, learning_rate=0.05, num_leaves=8, random_state=SEMILLA, n_jobs=2, verbose=-1)`	2
`svm_rbf`	`svm`	`SVC(kernel="rbf", C=1, probability=True, random_state=SEMILLA)` con `escalar=True`	1

Podéis añadir también `xgboost` (familia `ensemble_boosting`). La SVM suele ser la más lenta: eso cuenta en el criterio de coste.

- **Elegir las dos familias.** Mirad el leaderboard y buscad en las **dos más prometedoras** (por ejemplo, el bosque y *boosting*, como en `S08_05`). Una persona por búsqueda.
- **Buscar sin hacer trampas.** Usad `GridSearchCV` (prueba todas las combinaciones) o `RandomizedSearchCV` (prueba combinaciones al azar), con `cv=cv`, `scoring="roc_auc"` y `.fit(X_train, y_train)`. **Nunca `X_test`.** Como `hacer_pipe` llama `clf` al paso del modelo, los nombres de la rejilla llevan ese prefijo: `clf__max_depth`, `clf__n_estimators`…
- **Cuidar el tiempo.** Si una búsqueda pasa de 5 minutos, reducid las combinaciones (`n_iter` en la aleatoria, menos valores en la rejilla). Si varias combinaciones quedan casi empatadas, elegid la más sencilla (regla de «1 error estándar» de `S08_05`).
- **Guardar el mejor.** Registrad el modelo con los mejores hiperparámetros con `registrar(evaluar_modelo(...))`. Si usáis el **mismo nombre**, la fila se actualiza; si usáis uno nuevo (`random_forest_ajustado`), se añade (recordad el máximo de 8). Añadid también esos modelos al diccionario `catalogo` (más adelante): lo necesitará el paso 4.

Qué deberíais ver. Entre 6 y 8 filas, de al menos 4 familias (`referencia`, `lineal`, `arbol`, `ensemble_bagging`, `ensemble_boosting`, `svm`). El modelo ajustado suele quedar igual o algo mejor que el de partida, pero no siempre; si no

mejora, es un resultado válido y se cuenta.

Hecho cuando: hay entre 6 y 8 modelos de al menos 4 familias, la búsqueda está hecha en dos familias con validación cruzada sobre train y el `catalogo` incluye los modelos ajustados.

Paso 4 · Elegir el campeón (día 2, 45 minutos)

1. Calcular la puntuación global de cada modelo con vuestros pesos y ver quién gana.
2. Probar otros pesos (análisis de sensibilidad) y anotar si el campeón cambia.
3. Evaluar al campeón en el test, **una sola vez**, y guardar el resultado.
4. Escribir tres o cuatro líneas: por qué ganó, por qué no ganó otro y qué cambiaría la elección.

Cómo se hace

- **Puntuación global.** Definid el diccionario `pesos` con las claves `"rendimiento"`, `"explicabilidad"` y `"coste"` (en decimales que sumen 1, por ejemplo 0.5, 0.3 y 0.2) y calculad `lb_pond = puntuacion_global(lb, pesos)`. El campeón es la **primera fila** (`campeon`, el nombre del modelo). Pistas: código explicado del cuaderno.
- **Mirad la incertidumbre.** Un AUC de 0,81 no es mejor que uno de 0,80 si sus desviaciones son parecidas: la diferencia puede ser ruido. Si dos modelos empatan, es un resultado válido: elegid el más simple y explicadlo.
- **Sensibilidad.** Probad al menos otros dos conjuntos de pesos (el cuaderno trae `escenarios`: solo rendimiento, equilibrado y explicabilidad primero) y apuntad en una tabla qué campeón sale con cada uno. Si cambia fácilmente, decidlo: la recomendación depende de lo que valore el cliente.
- **El examen final.** Antes de seguir, comprobad en equipo que el campeón es el definitivo. Luego, en el cuaderno: reconstruid el campeón con `catalogo[campeon]()`, ajustadlo con **todo** `X_train`, calculad `auc_test_campeon` (su AUC sobre `X_test` con `roc_auc_score`) y guardadlo con `actualizar_test(campeon, auc_test_campeon)`. **No se evalúa ningún otro modelo en el test**, y el campeón no se cambia después de ver ese número.
- **Las líneas de la decisión.** Con el ejemplo de este modelo: «Campeón: … (AUC … ± …). No ganó … porque …. Si el cliente valorara solo …, elegiríamos …». Esas líneas van a la *model card*.

Qué deberíais ver. Una línea como `Campeón: … · AUC en validación cruzada 0.… · AUC en test 0.…` (el AUC de test está entre 0,65 y 0,85, y cerca del de validación; si está mucho más abajo, hay sobreajuste). En `leaderboard_S08.csv`, la columna `auc_test` tiene valor en **una sola fila**. Y una tabla de sensibilidad con el campeón de cada escenario.

Hecho cuando: hay un campeón elegido con vuestros pesos, la sensibilidad está probada y apuntada, y `auc_test` aparece en una única fila.

Paso 5 · *Model card*, README y entrega (día 2, 30 minutos)

1. Copiar `model_card_plantilla.md` como `model_card.md` y rellenarla con vuestros datos.
2. Escribir el `README.md`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado en el README.
4. Subir los cuatro ficheros a `sprint-08/`.

Cómo se hace

- **La *model card*.** Descargad la plantilla (en GitHub: *Raw*, copiar, y crear `model_card.md` con *Add file → Create new file*), o editadla en cualquier editor de texto. Debe caber en 1–2 páginas y cada apartado lleva **vuestros** datos (nada de dejar los ejemplos). Para el **nivel mínimo** bastan estos apartados, en pocas líneas:

Apartado de la plantilla	Qué escribís
1. Resumen	Qué predice el modelo, para qué decisión y quién lo usa
2. Modelo campeón	Familia, hiperparámetros finales y versiones de las librerías
3. Por qué este campeón	Vuestros pesos, el resultado del campeón, el mejor rival y las líneas del paso 4
5. Rendimiento	AUC en validación cruzada (± desviación) y **AUC de test (una sola evaluación)**

Apartado de la plantilla	Qué escribís
6. Uso previsto y NO previsto	Para qué sirve y para qué **no** debe usarse (por ejemplo, no denegar reservas ni decidir sin revisión humana)

Para el **nivel estándar**, completad también 4 (datos y límites: son sintéticos), 7 (cómo se explica una predicción a un gerente), 8 (riesgos: el **país de origen** es una variable sensible; qué pasa con una falsa alarma), 9 (mantenimiento: qué mediríais en producción para detectar la **deriva**, cada cuánto se reentrena y quién se encarga) y 10 (cómo reproducirlo).

- **El README.** Tres cosas que no pueden faltar: **qué se entrega y quién hizo qué**; **cómo se reproduce** (abrir `reto_S08.ipynb` en Colab y *Reiniciar y ejecutar todo*; versiones de `scikit-learn`, `xgboost` y `lightgbm`; `SEMILLA = 42`; orden de ejecución); y **uso de IA** (qué pedisteis y qué revisasteis; los datos son sintéticos, sin datos personales). Para saber las versiones, en una celda del cuaderno: `import sklearn, xgboost, lightgbm` y mostrad `sklearn.__version__`, `xgboost.__version__` y `lightgbm.__version__`.
- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica y añade al final del README esta sección, con un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».

```
## Revisión con la rúbrica
**Revisado por:** (nombre) · **Fecha:** (dd/mm/aaaa)

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** (una cosa concreta)
```

- **Preparar los ficheros.** En Colab: *Entorno de ejecución → Reiniciar y ejecutar todo* (así queda con los resultados visibles y sabéis que funciona de arriba abajo), después *Archivo → Descargar → Descargar .ipynb*. Descargad `leaderboard_S08.csv` desde el panel de archivos (clic derecho → *Descargar*) **antes de cerrar Colab**: al cerrar la sesión se pierde. Ojo: al reiniciar y ejecutar todo, el cuaderno vuelve a crear el CSV desde cero, así que descargadlo después.
- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-08`**; (2) *Add file → Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `reto_S08.ipynb` y `leaderboard_S08.csv`, la B sube `model_card.md` y la C sube `README.md`. Respetad los nombres tal como están en la tabla A (el reto fija `leaderboard_S08.csv` y `reto_S08.ipynb`, con la S mayúscula), sin espacios.

Qué deberíais ver. En la carpeta `sprint-08` del repositorio, los cuatro ficheros del equipo y una hoja de autoevaluación por persona (**4 + N**); y en el historial (el icono del reloj), *commits* con nombres distintos.

Hecho cuando: otra persona podría ejecutar el cuaderno siguiendo solo el README y obtendría el mismo campeón, la sección «Revisión con la rúbrica» está rellena y los cuatro ficheros están en `sprint-08/`.

Después, 35 minutos de entrega, review y retro: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-08/`. En la *review* (5 minutos por equipo) enseñáis el leaderboard (la tabla o un gráfico), el campeón y **por qué no ganó otro**, y una frase sobre qué cambiaría la elección si el cliente valorara otra cosa. Preguntas de la retrospectiva: ¿nos sorprendió que un modelo sencillo compitiera con uno complejo? ¿Qué criterio pesó más en nuestra decisión? ¿Qué mediríamos en producción para saber que el modelo se degrada?

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da respuestas que parecen correctas y no lo son**. Reglas:

- **La IA no decide el campeón:** lo decide el cálculo con vuestros pesos.

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea o por un número de la *model card*, tenéis que saber explicarlo.
- **Declaradlo** en el README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un modelo	«Explícame cómo funciona un bosque aleatorio con una analogía de votación de expertos, sin fórmulas»	Que podéis explicárselo al gerente
Entender un hiperparámetro	«¿Qué hace `max_depth` en un árbol? ¿Qué pasa si es muy grande?»	Que probáis valores y veis lo que ocurre (sobreajuste)
Redactar la *model card*	«Ayúdame a redactar la sección de limitaciones. Estos son los datos: *[resumen]*»	Que **todo** lo que escriba coincide con vuestros resultados
Preparar la defensa	«Hazme 5 preguntas difíciles que un gerente podría hacer sobre mi elección de campeón»	Que preparáis la respuesta con **vuestros** datos

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Comparar modelos con particiones distintas	Cada persona prueba «por su lado»	Usad el cuaderno plantilla y no cambiéis la celda de herramientas: mismos folds para todos
Ajustar hiperparámetros mirando el test	«Para ver si mejora»	Buscad solo con `X_train` y validación cruzada; `X_test` solo aparece en el paso 4
Evaluar varios modelos en el test y quedarse con el mejor	Curiosidad o prisa	El test se usa **una vez**, con el campeón ya elegido; `auc_test` en una sola fila
Elegir por el AUC más alto sin mirar la desviación	«0,81 es más que 0,80»	Si las desviaciones se solapan, la diferencia puede ser ruido: elegid el más simple
`model_card` genérica	Se copian los ejemplos de la plantilla	Cada apartado con **vuestros** datos y riesgos
Olvidar la explicabilidad y el coste	Solo se mira la precisión	Puntuadlos y ponderadlos con `puntuacion_global`
Búsquedas que duran horas	«Más combinaciones»	Máximo 5 minutos por búsqueda: menos combinaciones o `RandomizedSearchCV` con pocas iteraciones
Se pierde el leaderboard	Colab borra los ficheros al cerrar y `REINICIAR = True` vuelve a empezar de cero	Descargad `leaderboard_S08.csv` antes de cerrar y no editéis el CSV a mano
El campeón no está en `catalogo`	Se registró un modelo ajustado y no se añadió al diccionario	Añadid cada modelo ajustado a `catalogo` con el mismo nombre del leaderboard

Ayuda 3 · Si os atascáis

1. **No sabéis qué peso poner:** pensad en lo que diría el gerente. Si prioriza entender, sube la explicabilidad.
2. **La búsqueda tarda demasiado:** reducid el número de combinaciones o usad búsqueda aleatoria con pocas iteraciones.
3. **Dos modelos empatan:** es un resultado válido. Elegid el más simple y explicadlo.
4. **Falla una comprobación (`assert`) del cuaderno:** leed el mensaje, que dice qué esperaba, y repasad el ejercicio de esa celda y su código explicado. Preguntad al equipo y, después, al docente.